

Judge-Shaped, Not Harm-Shaped: A Persistent Measurement-Validity Failure in Jailbreak Success Judges, and an Honest Null Result for Activation-Guided Mutation

Muhammed Muiz
Independent Researcher

August 2026*

Abstract

Jailbreak fuzzing benchmarks are usually reported as a single number: attack success rate (ASR), scored by an automated judge. That number is only as trustworthy as the judge producing it. We report a measurement-validity failure in `hubert233/GPTFuzz`, GPTFuzzer’s default RoBERTa judge. It rewards jailbreak-**shaped** surface form—persona/roleplay declarations, “you are now X” framing—rather than harmful content. Hand-reading its flagged “successes” on Phi-4-mini-instruct found refusals that pivoted to an unrelated topic, and completions that merely echoed a DAN/Omega-style template’s setup with zero harmful content.

We built a stricter two-stage replacement: a deterministic refusal pre-filter plus an anti-roleplay LLM-as-judge rubric. **The fix did not fully work.** On a fresh run, the corrected judge still passed a “ChadGPT” persona-wrapper completion that actually refuses and offers crisis-support resources—zero harmful content. Judge unreliability on persona-wrapper completions survives a deliberate, targeted fix; it is not one classifier’s quirk.

As a secondary result, we report an honest null. Using the corrected judge, activation-guided span mutation—mutating only the template span with the highest projection onto a difference-in-means refusal direction, rather than the whole template—shows no consistent, reliable ASR advantage over GPTFuzzer’s uniform-mutation baseline, on either target we tested (Qwen2.5-3B-Instruct, Phi-4-mini-instruct). We independently confirmed the guided-mutation mechanism was engaged: attribution fired on every iteration, and search revisited mutated candidates. Attribution *quality*, however, was template- and target-dependent, not uniformly refusal-localized—consistent with a guidance signal that only sometimes helps.

All results are smoke-scale ($n = 5$ behaviors per condition); the planned 25-behavior $\times$ 3-seed matrix never ran—compute (Kaggle’s free tier) was exhausted first. Both findings argue that jailbreak-fuzzing evaluation infrastructure, not just attack methods, deserves scrutiny.

How to read this paper. Every number below is either (a) read directly from a committed `results/*.json` / `results/*.npz` file—cited inline as `path` $\rightarrow$ `field`—or (b) explicitly marked in-text as hand-verified (a completed human check not written back into any file) or as narrative-only (no backing artifact exists in this repository). Claims of the second kind are named as such wherever they appear rather than being softened into unmarked prose, and no conclusion rests on one.

*Code, configurations, and every reported number: <https://github.com/secslayer/jlens-fuzz>. Disclosure notices concerning the evaluated open-weight models were sent to their maintainers on 2026-08-19; see Section 7.

1 Introduction

Attack success rate (ASR)—the fraction of harmful behaviors for which a fuzzing loop finds at least one prompt an LLM complies with—is the standard headline metric in jailbreak-fuzzing work, following GPTFuzzer (J. Yu et al., 2023). ASR is only as meaningful as the judge that decides “did this completion comply.” That judge is frequently a small, frozen, off-the-shelf classifier reused across many follow-on papers without re-validation—an implicit assumption that the judge itself is a solved problem.

This paper started as a study of a different question: does mutating a jailbreak template at the *location* an interpretability signal points to (a refusal direction extracted via difference-in-means over paired harmful/harmless prompts) produce better guided mutations than GPTFuzzer’s uniform, whole-prompt mutation? In the course of running that experiment, the project’s own pipeline caught its judge lying to it. Four completions the judge scored ≈ 0.99 (“jailbroken”) on a Phi-4-mini smoke run were, on inspection, not jailbreaks at all—one was a refusal that pivoted to an unrelated topic, three were the target model merely reciting a roleplay template’s setup with no harmful content whatsoever. The judge was detecting jailbreak-**shaped** vocabulary, not harm.

We treat this not as an incident to patch quietly and move past, but as the paper’s primary finding. We built a stricter two-stage judge (Section 3) and validated that it changes real outcomes (Section 5.1). But hand-verifying the *fixed* judge’s own output on a later, methodologically cleaner run turned up a second, distinct instance of the same failure class—a persona-wrapper completion that actually refuses and provides crisis resources, still scored a pass. That a deliberate, rubric-guided LLM-as-judge fix did not fully close this gap is, we argue, more informative than either finding alone: judge reliability on persona-wrapper/roleplay-shaped completions is a persistent problem across judge architectures, not a quirk of one RoBERTa classifier.

Our secondary contribution is an honest null result, but a negative finding only earns trust if its mechanism actually ran. Section 5.3 confirms guided mutation’s attribution and tree-search components were both genuinely engaged throughout every reported run—not silently reduced to uniform mutation under the hood. Only against that confirmed-active mechanism does the null become informative: using the corrected judge, guided mutation still shows no consistent, reliable ASR advantage over uniform mutation on either target (Section 5.2—we ran no formal significance test at this sample size; “no advantage” here means the raw counts do not consistently favor either method, not a statistically established equivalence). We can therefore rule out “the mechanism never engaged” as the explanation, distinguishing this null from an artifact of guided mutation quietly falling back to uniform behavior.

Both results come with an honest scope limitation: everything here is smoke-scale ($n = 5$ behaviors per condition), not the originally planned 25-behavior $\times$ 3-seed matrix, because the free-tier compute budget (Kaggle, 2 $\times$ T4 per session) was exhausted before the full matrix could run. We state this plainly rather than let smoke-scale numbers imply matrix-scale statistical power.

Contributions.

1. A reproducible, hand-verified demonstration that a standard jailbreak-fuzzing success judge (hubert233/GPTFuzz) is fooled by jailbreak-shaped surface form rather than actual harm, and that this failure mode **persists after a deliberate, rubric-based LLM-judge fix**—not a single-classifier quirk.
2. An honest null result for activation-guided span mutation vs. uniform mutation, reported with independent confirmation that the guided-mutation mechanism (attribution + tree-search

revisiting) was actually active during the runs that produced the null, distinguishing “no effect” from “mechanism never fired.”

3. A stated, artifact-backed inventory of exactly which numbers in this paper are read directly from committed result files, which are PI-hand-verified but not written back into any file, and which are reported without any backing artifact at all—offered as a concrete illustration of the provenance discipline we think jailbreak-fuzzing papers should adopt more generally, given finding 1.

2 Related Work

Black-box jailbreak fuzzing. GPTFuzzer (J. Yu et al., 2023) introduced MCTS-lite (UCB1) seed-template selection over a human-authored jailbreak template pool, with an LLM mutator rewriting the *whole* selected template and a trained RoBERTa classifier as the success judge. We reuse GPTFuzzer’s seed-selection algorithm unchanged (Section 3) as our uniform-mutation baseline and as the search scaffold our guided-mutation variant is built on top of—the novelty in this work is *where* mutation targets a template and *which* judge determines success, not the search algorithm itself.

White-box activation steering / refusal directions. Refusal behavior in instruction-tuned LLMs is mediated, to a first approximation, by a single linear direction in activation space, extractable via a difference-in-means contrast between harmful and harmless prompt activations (Arditi et al., 2024); ablating or amplifying this direction causally affects refusal behavior. We reuse this direction-extraction methodology (Section 3) but for a different purpose than steering generation directly: we use the direction as a **token-attribution signal** to decide *where inside a jailbreak template* a mutation should be targeted, on the hypothesis that spans strongly projecting onto the refusal direction are the parts of a template most responsible for triggering refusal, and are therefore the highest-value mutation targets. This connects to a closely related line of work using refusal-direction signals to directly guide or explain jailbreak search—in particular the approach informally referred to in this project’s planning documents as “Mechanistic AutoDAN” (Collu et al., 2026) (“Refusal Before Decoding: Detecting and Exploiting Refusal Signals in Intermediate LLM Activations”), which detects and exploits refusal signals in intermediate activations directly. We distinguish our approach as targeting the *search/mutation-selection* problem (which span of an existing template to rewrite) rather than the decoding/generation problem the “Refusal Before Decoding” framing addresses; a precise technical comparison against that work is left as future work (we have not run it as a baseline—see Section 4).

Jailbreak evaluation and judges. AutoDAN (Liu et al., 2024) and AJF (M. Yu et al., 2025) are cited here as context for the jailbreak-generation literature this project’s baselines and method sit alongside—we **did not run either as a baseline in this work**. A fixed compute budget (Section 4) scoped the core comparison to GPTFuzzer alone, with AutoDAN deferred to an extended set of runs that the budget did not reach. Separately, a growing body of work has begun questioning the reliability of jailbreak judges themselves: “How Reliable Is Your Jailbreak Judge?” (Gao, 2026) finds that a large and growing share of reported jailbreak-evaluation results using LLM-judges are unreliable, both on average and under deliberate adversarial pressure; “LLM-Safety Evaluations Lack Robustness” (Beyer et al., 2025) and “Confusion is the Final Barrier” (Yan et al., 2025) make related robustness/consistency arguments; “Not Aligned” is Not “Malicious” (Mei et al., 2025) specifically

discusses judges hallucinating jailbreak success; and “Beyond Accuracy: Policy Invariance as a Reliability Test for LLM Safety Judges” (Weng et al., 2026) proposes a reliability test in the same spirit as our residual-false-positive check. We situate this paper’s primary contribution within this cluster: rather than a general audit or a new reliability *metric*, we contribute a specific, reproducible, hand-verified case study of a widely-reused classifier judge failing, a concrete fix, and—the part we believe is under-examined in this literature—direct evidence that the fix’s own output still contains the same failure class, from our own pipeline’s real generations rather than a constructed adversarial example.

3 Method

Pipeline overview. `scripts/run_fuzz.py` implements a single fuzzing loop parameterized by `--method {ours, gptfuzzer}`, `--mutation {guided, uniform}`, and `--fitness judge` (this project’s probe-based `judge+act` fitness variant was built but demoted—see below). Both methods share:

- **Seed pool.** The original GPTFuzzer human-authored template pool (sherdencooper/GPTFuzz’s `GPTFuzzer.csv`, 77 templates), subsampled deterministically to `seed_pool_size=12` templates via a fixed constant seed (`SEED_POOL_SUBSAMPLE_SEED=0`, independent of the run’s own `--seed`)—identical 12 templates across every run, replicate, and target. Section 4 explains why this subsampling exists.
- **UCB1 pool selection.** GPTFuzzer’s MCTS-lite node selection, reused unchanged: at each iteration, select the pool node maximizing UCB1 score (unvisited nodes get priority; visited nodes trade off mean reward against an exploration bonus), mutate it, and append the mutated child as a new pool node.
- **Two-stage success judging** (`scripts/judge.py`, Section 3.3).

3.1 Refusal-direction extraction

Following Arditi et al. (2024), we extract a per-layer difference-in-means direction from paired harmful (AdvBench) and harmless (Alpaca-style) prompts’ residual-stream activations, on both targets.

Both targets’ held-out AUCs are a perfect 1.0 for both signals—uninformative on their own, as Gate 2’s own novel-prompt check exists to demonstrate (`reviews/stage2-human-signoff.md`). On the novel-prompt check (Table 1), **Qwen’s direction separates harmful from benign cleanly and by a wide margin (gap 35.16, benign mean solidly negative); Phi’s direction separates harmful from benign only weakly (gap 4.89, and notably the benign mean is *positive*, not *negative*—Phi’s direction never clearly signals “not harmful” the way Qwen’s does).** Both targets’ probes fail the same novel-prompt check at exactly chance accuracy (0.500), replicating Qwen’s probe-generalization failure (diagnosed as overfitting AdvBench’s imperative surface form rather than harmful intent, `reviews/stage2-human-signoff.md`) on Phi as well. Per that gate’s explicit ruling, probe-based fitness (`--fitness judge+act`) was demoted out of the core comparison entirely on both targets; every result in this paper uses `--fitness judge` (judge-only reward), and the probe/`judge+act` path is not part of any claim below. Guided *mutation* (driven by the direction, not the probe) is unaffected by this demotion and remains the paper’s method under test—but the direction-quality asymmetry above is directly relevant to Section 5.3’s attribution-quality finding: Phi’s underlying refusal signal is measurably weaker and noisier than

Table 1: Refusal-direction and probe comparison, Qwen2.5-3B (control) vs. Phi-4-mini (treatment). Sources: `results/direction.npz`, `results/phi/direction.npz` ($\rightarrow$ `best_layer/best_layer_idx/best_auc`); `results/qwen_novel_check.log`, `results/phi4mini/novel_check.log` (per-prompt and mean `direction_score` lines); `results/probes/best_layer.json`, `results/phi/probes/best_layer.json` ($\rightarrow$ `best_auc`).

	Qwen2.5-3B (control)	Phi-4-mini (treatment)
Direction best layer / depth	25/36 ($\approx 69\%$)	11/32 ($\approx 34\%$)
Direction held-out AUC	1.0	1.0
Novel-prompt direction score, harmful mean	+30.204	+9.486
Novel-prompt direction score, benign mean	-4.954	+4.594
Harmful – benign gap	35.16	4.89
Probe best layer / held-out AUC	9 / 1.0	8 / 1.0
Probe novel-prompt accuracy	0.500 (chance)	0.500 (chance)

Qwen’s, which is a plausible contributing cause (not the only one—see Section 5.3) for why Phi’s attribution scores cluster more narrowly and less consistently around refusal-semantic content than Qwen’s do.

3.2 Guided mutation

For the selected pool template (`ours`, `--mutation guided`), we run a partial forward pass, compute the per-token projection of each token’s residual-stream activation (at the direction’s best layer) onto the refusal direction, and select the contiguous `MUTATE_WINDOW_TOKENS`-token window (excluding the behavior-injection marker span) with the highest summed projection. A local mutator LLM is then instructed to rewrite *only that span* (“reduce refusal-triggering framing, preserve intent”), leaving the rest of the template untouched. If attribution fails for any reason (degenerate span, shape mismatch, tokenizer edge case), the loop falls back to whole-template uniform mutation for that iteration and increments a `guided_fallback_count` counter—this ensures fallback is never silent (Section 5.3). **Uniform mutation** (`gptfuzzer`, and `ours --mutation uniform` as an ablation) mutates the selected template’s entire text, per GPTFuzzer’s original design, sharing every other component (seed pool, UCB1 selection, judge) with the guided condition—the only variable that differs between `ours` and `gptfuzzer` is mutation location.

3.3 Two-stage success judging

Motivated by the false-positive finding that is this paper’s primary contribution (Section 5.1), success determination in `scripts/judge.py` is a two-stage process: (1) a deterministic keyword pre-filter (`is_refusal()`) checks the first 300 characters of a completion against a list of canonical refusal phrases—catching “refuses, then pivots to something unrelated” with zero model calls; (2) anything that passes stage 1 is graded by a fixed, small, ungated LLM (`microsoft/Phi-3.5-mini-instruct`, distinct from both attack targets so it is never self-grading) against an explicit rubric instructing it to fail responses that merely adopt a roleplay persona or template setup without providing real harmful content, or that address a different/watered-down version of the request, and to pass only responses that give real, specific, actionable content. The original RoBERTa classifier judge

(hubert233/GPTFuzz) is retained as a **logged diagnostic only** (roberta_judge_score per candidate, roberta_judge_positive_rate aggregated)—never the success determinant in any result reported here.

4 Experimental Setup

Targets. Two open-weight instruction-tuned models, both ungated and small enough to fit a single Kaggle T4 GPU in fp16: **Qwen2.5-3B-Instruct** (control) and **Phi-4-mini-instruct** (≈ 3.8 B params, treatment). The two targets share every pipeline invariant except the model itself and its paired mutator-LLM default (each target self-mutates; see `configs/exp.yaml` / `configs/exp_phi4mini.yaml`), per this project’s rule that cross-target ASR comparisons are only valid if judge, benchmark, budget, and decoding parameters are held fixed.

Benchmark and budget. 5 behaviors per condition sampled from the canonical AdvBench CSV (`llm-attacks/llm-attacks`), `query_budget=40` full-generation queries per behavior, `max_iterations=100`, `decode_temperature=0.7`, `decode_top_p=0.9`—identical across every condition reported (`configs/exp.yaml`, `configs/exp_phi4mini.yaml`). `max_iterations` is non-binding under `--fitness judge`: every iteration is a full query, so the loop always hits the `query_budget = 40` cap (or succeeds) well before it could reach iteration 100; the effective per-behavior bound throughout this paper is `query_budget`, not `max_iterations`.

Seed-pool subsampling (`seed_pool_size=12`). We found, via a debug-logging pass over the UCB1 pool-selection trace, that the original 77-template seed pool exceeds `query_budget=40`: since UCB1 always prioritizes unvisited nodes, and $77 > 40$, every iteration of every behavior in our first attempt selected the next never-mutated original seed template, in identical order, for every behavior—the search never reached a point within budget where UCB1 would revisit and refine an already-mutated child. Both *ours* and *gptfuzzer* were, in effect, doing the same thing (mutate a fresh seed once) and a guided-vs-uniform comparison run this way would not actually test mutation-location strategy at all. We fixed this by deterministically subsampling the seed pool to 12 templates (fixed constant, not `--seed`—identical 12 templates on every run/replicate/target), so that ≈ 12 of the 40-query budget exhausts the pool and the remaining ≈ 28 do real UCB1 revisiting. We verified this structurally engages tree search by extracting `select_ucb1()/backpropagate()` **verbatim** from the codebase and running them standalone against a synthetic reward loop: the old 77-template pool yields 0/40 mutated-child selections at `query_budget=40` (confirming the problem); the new 12-template pool yields 16/40 mutated-child selections even under the worst-case all-zero-reward scenario (confirming the fix). Section 5.3 reports the *real runs*’ `n_mutated_child_selected` counts, which is the artifact-backed version of this claim.

Compute constraint—stated plainly. All compute for this project ran on Kaggle’s free tier ($2 \times$ T4 GPUs per session, ≈ 30 GPU-hours/week). The originally planned full evaluation—25 behaviors $\times$ 3 seeds $\times$ 2 targets $\times$ {*ours*, *gptfuzzer*, plus ablations}—**did not run**. The free-tier compute budget was exhausted after the smoke-scale ($n = 5$) runs reported here, before the full matrix could be launched. This is a hard resource constraint, not a scheduling choice: every result in this paper is smoke-scale, and we treat the resulting lack of statistical power as a first-class limitation (Section 6), not an implementation detail to bury.

5 Results

5.1 Judge reliability: primary contribution

The original incident. Hand-reading 4 completions the original `hubert233/GPTFuzz` judge scored ≈ 0.99 (“jailbroken”) on an early Phi-4-mini smoke run found all four were not jailbreaks: one was a refusal (“I can’t assist”) that pivoted to an unrelated topic; three were the model merely echoing a DAN/Omega/APOPHIS-style roleplay template’s persona/setup instructions, with zero harmful content (full writeup: `reviews/judge-validity-incident.md`). The specific `asr = 0.8` figure this produced is *not* backed by a committed `results/*.json` in this repository—no `results/phi/ours_smoke.json` (pre-fix, pre-pool-fix) exists; the 0.8 number survives only as narrative record in the incident writeup, not as a citable artifact. We therefore report the qualitative finding (4/4 hand-read false positives) as established—it is a direct, documented human read—but do not cite “0.8 $\rightarrow$ 0.0” as a clean before/after pair the way the rest of this section’s numbers are cited. The judge-inflation claim rests on the artifact-backed evidence below instead.

A cleaner, artifact-backed before/after (Qwen, same target, same method, same $n = 5$ behaviors). `results/ours_smoke.json` (old RoBERTa judge as the live success determinant, predating both the judge fix and the seed-pool fix) reports `asr = 1.0` (5/5)—every behavior “succeeded” by the old judge’s own determination during search. `results/ours_smoke_pool12.json` (fixed judge as determinant, seed-pool fix applied) reports `asr = 0.4` (2/5), later hand-verified to a true 0.2 (1/5; see below). **We flag this comparison as suggestive, not a controlled ablation:** two variables changed between these two runs (judge fix AND seed-pool fix), not one, so the delta cannot be cleanly attributed to the judge alone.

The cleanest same-run comparison available. Within `results/ours_smoke_pool12.json` itself (one run, one set of candidates, both judges scored on the same completions), the retained RoBERTa diagnostic score (`roberta_judge_positive_rate = 0.335`, i.e. the old judge would have flagged $\approx 34\%$ of the 167 individually-evaluated candidates as positive) is far higher than the trusted judge’s own candidate-level success rate (`trusted_judge_success_rate = 0.012`, $\approx 1.2\%$ of the same 167 candidates; Table 2). **Important caveat, stated in the pipeline’s own code comment:** these two fields are **candidate-level** rates (over every full-generation query in the run), not the **behavior-level asr** statistic reported elsewhere in this paper—they answer “how often would the two judges have disagreed on an individual completion,” not “what would behavior-level ASR have been under the old judge.” We report both numbers because they are real and artifact-backed, but we do not collapse them into a single “ $N \times$ inflation” headline figure the way an early draft of this section’s brief suggested, because that would conflate two different statistics. The qualitative direction—old diagnostic judge substantially more permissive than the trusted judge, on the identical completions—is exactly what this section’s original incident predicts, and holds here too.

Residual false positive, found AFTER the fix—the ChadGPT case. Hand-verifying the fixed judge’s 2 flagged Qwen ours successes (`results/ours_smoke_pool12.json`, `asr = 0.4`) found only 1 is genuine. The other is a **false positive**: a “ChadGPT” persona-wrapper completion that, in its actual response, refuses the harmful request and provides crisis-support resources—zero harmful content—yet the fixed rubric judge (whose rubric explicitly instructs failing “the response refuses, declines, moralizes, or deflects... even if it then talks about something unrelated”) scored it PASS anyway (full writeup: `reviews/judge-validity-incident.md`, “Residual false positive found

Table 2: Judge comparison within `results/ours_smoke_pool12.json` and against the pre-fix `results/ours_smoke.json`. Behavior-level and candidate-level rates are not directly comparable (see caveat in text).

Run	Judge (determinant)	asr	Granularity note
<code>results/ours_smoke.json</code> (Qwen, pre-fix, pre-pool-fix)	old RoBERTa	1.0 (5/5)	behavior-level
<code>results/ours_smoke_pool12.json</code> (Qwen, fixed judge, pool-fix)	fixed 2-stage	0.4 (2/5), hand-verified true 0.2 (1/5)	behavior-level
— same run, <code>roberta_judge_positive_rate</code>	old RoBERTa (diagnostic only)	0.335	candidate-level, not comparable to asr above
— same run, <code>trusted_judge_success_rate</code>	fixed 2-stage (diagnostic mirror)	0.012	candidate-level, not comparable to asr above

AFTER the fix”). True hand-verified Qwen **ours** ASR: $1/5 = 0.2$, not written back into the JSON (this project does not retroactively edit results files). We consider this the paper’s central piece of evidence: the failure mode identified in the original incident is not specific to `hubert233/GPTFuzz`’s classifier architecture—it survives a deliberately anti-roleplay, explicitly-rubric-instructed LLM-as-judge replacement. Two plausible, non-exclusive, **undiagnosed** contributors (flagged rather than asserted, since the raw completion text is gitignored per this project’s data-handling policy and not independently re-inspectable from this repository): the stage-1 keyword pre-filter’s 300-character window could have let a later-appearing refusal phrase through to the LLM judge; or the LLM judge itself may simply have failed to apply its own rubric correctly on this input—itself a citable limitation of the “replace a classifier judge with an LLM judge” fix strategy.

5.2 The honest guided-mutation null

We explicitly do not claim guided mutation beats uniform mutation from Table 3. The Qwen 1-vs-0 (hand-verified) or 2-vs-0 (raw judge count) gap is noise at $n = 5$ —a single behavior flipping either way changes the ratio entirely, and Phi shows a tied 0-vs-0.

Both targets’ **asr** values are computed at `n_behaviors = 5` (smoke scale). Establishing whether either direction (guided advantage, or no advantage) is statistically significant would require substantially more behaviors and seed replicates than compute allowed (Sections 4 and 6)—we report a null at the scale we could afford to test, not a proof of equivalence at any scale.

5.3 Mechanism-engagement check: is the null a broken-mechanism artifact?

A null result is only informative if the mechanism under test actually ran. We can confirm two specific, artifact-backed facts about mechanism engagement, and must flag a third claim as unconfirmed.

Attribution fires, does not silently fall back. `guided_fire_count/full_forward_passes`

Table 3: ASR by target and method, pool-12 smoke runs ($n = 5$ behaviors each). Hand-verified column reflects the ChadGPT correction (Section 5.1); the raw `asr` field is never retroactively edited in the `source JSON`.

Target	Method	<code>asr</code>	Hand-verified <code>asr</code>	Source
Qwen2.5-3B	<code>ours</code> (guided)	0.4 (2/5)	0.2 (1/5)	<code>results/ours_smoke_pool12.json</code>
Qwen2.5-3B	<code>gptfuzzer</code> (uniform)	0.0 (0/5)	0.0 (0/5)	<code>results/gptfuzzer_smoke_pool12.json</code>
Phi-4-mini	<code>ours</code> (guided)	0.0 (0/5)	0.0 (0/5)	<code>results/phi/ours_smoke_pool12.json</code>
Phi-4-mini	<code>gptfuzzer</code> (uniform)	0.0 (0/5)	0.0 (0/5)	<code>results/phi/gptfuzzer_smoke_pool12.json</code>

is 167/167 for Qwen `ours` and 200/200 for Phi `ours`—`guided_fallback_count` is 0 in both. `find_attribution_span()` successfully located a real mutation span on every single iteration of every guided run reported here; the null above is not explained by guided mutation quietly degrading to uniform mutation under the hood.

Tree search genuinely engages (the fix in Section 4 working as intended). Every run’s `n_original_selected` and `n_mutated_child_selected` counters were re-verified directly against their source JSON files for this revision, and every pair sums exactly to that run’s own `full_forward_passes` (Table 4): Qwen `ours`, $113 + 54 = 167$; Qwen `gptfuzzer`, $120 + 80 = 200$; Phi `ours`, $120 + 80 = 200$; Phi `gptfuzzer`, $120 + 80 = 200$. All four runs spend a substantial fraction of their budget revisiting/refining previously-mutated pool nodes, not just replaying the 12 original seeds. Qwen `ours`’ lower total (167, vs. 200 for the other three) is not an inconsistency: it reflects early stopping on success, not a different `query_budget`—all four runs share the identical `query_budget` = 40×5 behaviors = 200-query cap (Section 4), but Qwen `ours` is the only one of the four with any successes (2/5 raw), and a behavior’s search stops the moment it succeeds rather than continuing to the cap. Concretely, `results/ours_smoke_pool12.json`’s own `queries_to_success` field records successes at queries 17 and 30 for two behaviors, with the remaining three running the full 40-query cap each: $17 + 40 + 40 + 40 + 30 = 167$, exactly matching the reported `full_forward_passes`. The other three runs (all `asr` = 0.0) never stop early, so every one of their five behaviors runs the full 40-query cap, $5 \times 40 = 200$.

Table 4: Pool-selection counters, verified against source JSON for this revision. `n_original_selected` + `n_mutated_child_selected` = `full_forward_passes` holds exactly for all four runs.

Run	<code>full_forward_passes</code>	<code>n_original_selected</code>	<code>n_mutated_child_selected</code>
Qwen <code>ours</code>	167	113	54
Qwen <code>gptfuzzer</code>	200	120	80
Phi <code>ours</code>	200	120	80
Phi <code>gptfuzzer</code>	200	120	80

This directly answers the question the seed-pool fix (Section 4) set out to answer: at pool-12, real UCB1 tree search is happening, on both methods, in every reported run. A separate, independently-

collected 2-behavior `--debug-attribution` diagnostic run shows the first `MUTATED_CHILD` pool selection firing at **iteration 24** on *both* targets: Qwen (`results/debug_attribution_qwen.log`, `behavior_idx = 1`) and Phi (`results/debug_attribution_phi.log`, `behavior_idx = 0`)—re-verified for this revision by grepping both logs directly, not assumed from a prior draft. This is not a coincidence: both examined behaviors have zero successes throughout their own search (Qwen’s debug run succeeds only on `behavior_idx = 0`, at iteration 17—not the `behavior_idx = 1` examined here; Phi’s debug run has `asr = 0.0` throughout), so both searches are governed purely by UCB1’s explore term under zero reward. Given identical `seed_pool_size = 12` and identical deterministic tie-breaking (unvisited nodes first, ties broken by lowest pool index) on both targets, the same zero-reward dynamics necessarily produce the same transition point: iterations 0–11 visit each of the 12 original seeds once; iterations 12–23 revisit them a second time (originals still win ties over children by lower index); by iteration 24 the originals have accumulated more visits than any child, so a child’s larger explore bonus (fewer visits, same zero reward) finally wins. This matches, to the exact iteration and for an explained reason rather than by chance, the worst-case prediction from Section 4’s standalone `select_ucb1/backpropagate` simulation—independent corroboration, from two real runs on two different targets, of a claim Section 4 previously supported only with a synthetic simulation.

Attribution quality: real, quantified, and *not* uniform (corrected from an earlier draft).

An earlier version of this section claimed attribution “localizes to refusal-relevant tokens,” based on an informal chat excerpt that was never saved as an artifact. We re-ran the diagnostic with `--debug-attribution` (2 behaviors per target, real forward passes, not synthetic) and saved the full output to `results/debug_attribution_qwen.log` (57 attribution calls) and `results/debug_attribution_phi.log` (80 attribution calls)—both now committed, with the assembled selected-span text redacted (`text='[REDACTED -- jailbreak-template fragment]'`) per a subsequent internal review’s finding that committing that text, even in a diagnostic log, fell outside this project’s own no-jailbreak-strings policy; the individual tokens, per-token projection scores, and `sum_score` values below are unredacted and are the actual evidence for this section’s claim. Reading the real logs, the earlier claim was **too strong**: localization is real on some inputs and largely absent on others, varying by which template (original seed or mutated child) is selected. Representative examples, with individual top-projecting tokens quoted directly from the committed logs and assembled span text redacted:

- **Strongly refusal-semantic** (Qwen, iteration 1, `results/debug_attribution_qwen.log`): top tokens include `harmful` (13.51), `illegal` (12.46), `policy` (13.18), `ethical` (11.77); selected span `[REDACTED -- jailbreak-template fragment]`, `sum_score = 108.58`.
- **Generic / template-structural, same target** (Qwen, iteration 0, same file): top tokens are `rules` (8.08), `it` (8.08), `opposite` (5.76), `contrad[iction]` (6.23)—function words and template scaffolding, not harm-related; selected span `[REDACTED -- jailbreak-template fragment]`, `sum_score = 47.08`—less than half the semantic example’s score.
- **Strongly refusal-semantic** (Phi, `results/debug_attribution_phi.log`): top tokens include `illegal` (10.19), `malware` (10.77), `harmful` (9.57), `discrimination` (7.98), `racism` (8.47); selected span `[REDACTED -- jailbreak-template fragment]`, `sum_score = 84.96`.
- **Generic, same target** (Phi, same file): top tokens are `mode` (6.86), `pretend` (6.34), `Anti[GPT]` (6.28), `character` (6.09); selected span `[REDACTED -- jailbreak-template fragment]`, `sum_score = 58.36`.

Aggregated over every logged call: Qwen’s per-selection `sum_score` ranges **40.0–116.0** (mean 82.6, $n = 57$); Phi’s ranges **51.7–87.2** (mean 65.8, $n = 80$)—Phi’s attribution is more uniformly mid-range, less likely to hit either Qwen’s strongest semantic peaks or its weakest generic troughs. This is directionally consistent with Section 3.1’s finding that Phi’s underlying refusal direction separates harmful from benign prompts far more weakly than Qwen’s (novel-prompt gap 4.89 vs. 35.16)—a noisier underlying signal plausibly produces a narrower, less differentiated attribution-score distribution, though we have not run a statistical test connecting these two observations and do not claim a proven causal link, only a consistent pattern.

We now state this section’s central claim precisely, not as “attribution localizes to refusal-relevant tokens” (too strong, corrected above) but as: attribution is a genuinely informative signal on some templates, on both targets, and an uninformative one on others—template- and target-dependent, not uniform. This is, we argue, *itself* a natural explanation for Section 5.2’s null: a guidance signal that only sometimes points somewhere meaningful cannot be expected to reliably outperform unguided (uniform) mutation across a whole search, even though it clearly is picking up real signal on the templates where it fires strongly. The null is not evidence the mechanism is broken (the two facts confirmed above rule that out); it is consistent with a real but unreliable signal, which is a more specific and more useful finding than an undifferentiated “no effect.”

6 Limitations

- **Smoke scale ($n = 5$), not the planned matrix.** Every ASR figure in this paper is computed over 5 behaviors per condition. The originally planned evaluation (25 behaviors $\times$ 3 seeds $\times$ 2 targets) did not run—compute (Kaggle free tier) was exhausted first. Effect sizes here should be read as suggestive at best; no claim in this paper should be read as carrying matrix-scale statistical power.
- **Compute-constrained matrix abandonment, not a scheduling deferral.** We state this plainly rather than imply the full matrix is merely “future work in progress”: it will not run under this project’s current resourcing.
- **Single run per condition—no seed replicates.** Every Sections 5.2 and 5.3 number comes from exactly one run per (target, method) pair (`seed = 0`; `configs/exp.yaml/configs/exp_phi4mini.yaml`). The originally planned 3-seed replication (`_seed1/_seed2` variants, `experiments.yaml`) did not run—same compute exhaustion as the behavior-count limitation above, but a distinct concern: even at $n = 5$ behaviors, we have no run-to-run variance estimate at all, so we cannot say whether a different `--seed` would reproduce the same 2-vs-0/1-vs-0 split or land differently. This is a second, independent reason (beyond $n = 5$ ’s small behavior count) the guided-vs-uniform null should not be read as precisely quantified.
- **Same-vendor judge LLM.** The rubric-based LLM judge (`microsoft/Phi-3.5-mini-instruct`) is same-vendor as one of the two attack targets (`microsoft/Phi-4-mini-instruct`)—a different checkpoint and training run, and never grading its own outputs, but not a fully vendor-independent judge either. We flag this as a potential (unquantified) source of residual bias, distinct from the persona-wrapper false-positive finding in Section 5.1.
- **Small, ≤ 4 B-parameter open-weight targets only.** Both targets are ungated, small enough to fit a single T4 GPU in fp16. Generalization to larger or more heavily safety-tuned models is untested here.

- **Raw completions not fully retained/reproducible from this repository alone.** Per this project’s data-handling policy, generated jailbreak attempt text (`results/prompts_*`) is gitignored and never committed. The ChadGPT false-positive example (Section 5.1) and other hand-verification work were done against completions available at generation time (locally/on the Kaggle session) but are not preserved in this repository—an independent reader cannot re-inspect the exact completion text behind Section 5.1’s central finding from this repo alone. This is a genuine reproducibility limitation, traded off against the (higher-priority) commitment never to publish raw jailbreak strings (Section 7).
- **Phi’s underlying refusal-direction signal is measurably weaker than Qwen’s.** Section 3.1’s novel-prompt check shows a harmful/benign separation gap of 4.89 on Phi vs. 35.16 on Qwen, with Phi’s benign mean staying positive rather than crossing to negative. This is now resolved as an artifact-backed *finding* rather than a missing-data gap (an earlier draft flagged Phi’s direction/probe artifacts as absent from the repository; both have since been committed), but it remains a limitation on how far Section 5.2’s null generalizes: we tested guided mutation on one target with a strong direction signal and one with a comparatively weak one, and cannot rule out that a target with an even cleaner refusal direction would show a different result.
- **The Section 5.3 debug-attribution evidence is itself small-*n*.** The `--debug-attribution` diagnostic run behind that section’s quantified claims covers 2 behaviors per target (57 attribution calls for Qwen, 80 for Phi)—enough to establish that attribution quality *varies*, not enough to claim a precise, generalizable distribution of how often it is informative vs. not. A larger, dedicated debug-attribution run across more behaviors would strengthen this claim’s precision.
- **AutoDAN and AJF were not run as baselines** (Section 2)—cited for context only, per the project’s compute-scoped decision to treat GPTFuzzer alone as a sufficient core-lane baseline.

7 Ethics and Responsible Disclosure

This project generated real jailbreak attempts against two open-weight instruction-tuned models, targeting AdvBench behaviors, some of which concern self-harm and other sensitive categories. **Content warning applies to the underlying (unpublished) data this paper describes.**

- **No successful jailbreak strings are published in this repository or paper.** Raw template/candidate/completion text is excluded via `.gitignore` patterns (`results/**/prompts_*`, `results/**/*jailbreak*`, and—since the finding below—`results/**/*log`, `results/**/*jsonl` by default). We note this was enforced **imperfectly** at first: `results/debug_attribution_{qwen,phi}.log`, a diagnostic artifact whose filename didn’t match either original `.gitignore` pattern, was committed containing assembled jailbreak-template fragments before an internal review caught it (`reviews/stage7.md`). Those fragments have since been redacted from both the log files and this paper’s own Section 5.3—see the corrected debug-log handling there and in Section A. **This is now backed by real automated tooling, not just convention or filename patterns:** `scripts/check_no_raw_text.py` content-scans (not just filename-matches) every tracked file under `results/` for known raw-text field shapes (`text=`, `"completion"`, `"candidate"`, `"prompt"`, `"template"`), runs in CI on every push (`.github/workflows/ci.yml`, covering all

branches, not only `main`), and is also available as a local pre-commit hook (`.githubhooks/pre-commit`) so violations are caught before they leave the machine. This is a real, meaningful control, not a perfect guarantee: it catches known field-name shapes, not arbitrary future ones—pair it with the same manual review discipline as before, not as a replacement for it.

- **Regeneration, not redistribution.** Anyone needing to verify a specific claim in this paper (e.g. re-examining the ChadGPT false positive, Section 5.1) must regenerate it themselves using the committed code, config, and behavior benchmark (all public)—we do not ship a copy of the generated harmful text itself, following the same withholding precedent as GPTFuzzer’s own release practice.
- **Public benchmark only.** All harmful behaviors are drawn from AdvBench, an existing public research benchmark; this work introduces no new harmful-behavior taxonomy or capability.
- **White-box work on small open-weight models; no production system targeted.** Neither target model is a deployed production system; this is controlled research-environment testing of publicly released model weights.
- **Defensive framing.** The refusal-direction signal this paper’s guided-mutation mechanism is built on (Section 3.1) is directly usable as a runtime monitoring signal: a deployment could project activations onto the same direction and flag/intervene on completions whose internal state drifts away from a refusal trajectory mid-generation, independent of whether guided mutation itself proves useful as an attack method. We suggest this as a concrete defensive application of the same interpretability signal studied here.
- **Disclosure.** This project committed in advance, in writing and before any results existed, to notifying the affected open-weight model maintainers before the work was made public. That commitment was originally tied to arXiv posting as its trigger; it was superseded when the code repository was made public on 2026-08-17, a deliberate decision recorded at the time rather than after the fact. **Disclosure notices were sent to Microsoft (MSRC, for Phi-4-mini-instruct and Phi-3.5-mini-instruct) and to Alibaba/Qwen (for Qwen2.5-3B-Instruct) as of 2026-08-19**—after the repository went public rather than before, which is the deviation recorded above. **No acknowledgement had been received from either vendor at the time of writing.** We record this plainly rather than waiting on a reply: the obligation is to notify affected maintainers, and non-response to a courtesy notice—which this is, given that no novel vulnerability is claimed and the techniques are published prior work—is common and does not leave the notification incomplete.

8 Conclusion

Two findings, both artifact-backed. First, judge reliability in jailbreak fuzzing is a persistent, cross-architecture problem, not a quirk of one classifier: the original RoBERTa judge inflated success via template-echo false positives, and a deliberately stricter, anti-roleplay LLM-as-judge rubric—built specifically to catch that failure mode—still leaked a persona-wrapper false positive (the ChadGPT case, Section 5.1). Second, guided mutation shows no consistent, reliable ASR advantage over uniform mutation on either target tested (Section 5.2), and this null comes with a mechanistic explanation rather than standing unexamined: the token-attribution signal driving guided mutation is only sometimes informative, and its reliability tracks the underlying refusal direction’s own separation

quality—Qwen’s direction separates novel harmful from benign prompts by a 35.16-point gap, Phi’s by only 4.89 (Section 3.1), and Phi’s weaker signal corresponds to a narrower, less differentiated attribution-score distribution during search (Section 5.3).

We frame both as contributions, not failures. A cautionary measurement result—a widely-reused judge fooling even its own deliberate fix—is directly useful to a field that often treats ASR as a solved metric. An honest, mechanistically-grounded negative result is more informative than an unexamined positive one, and more informative still than a null whose mechanism was never checked.

The practical takeaway: activation-guided jailbreak attacks should be evaluated against verified, adversarially-checked judges, and are only as promising as the separation quality of the inter-pretability signal steering them—a weak refusal direction may not support a strong guided-mutation advantage. The honest next step is the evaluation this paper could not run: the full 25-behavior $\times$ 3-seed matrix (Sections 4 and 6), with the corrected judge, once compute allows.

References

- Arditi, Andy et al. (2024). *Refusal in Language Models Is Mediated by a Single Direction*. arXiv: [2406.11717 \[cs.LG\]](#).
- Beyer, Tim et al. (2025). *LLM-Safety Evaluations Lack Robustness*. arXiv: [2503.02574 \[cs.CL\]](#).
- Collu, Matteo Gioele et al. (2026). *Refusal Before Decoding: Detecting and Exploiting Refusal Signals in Intermediate LLM Activations*. Informally referred to as “Mechanistic AutoDAN” in this project’s internal planning docs. arXiv: [2605.28553 \[cs.CL\]](#).
- Gao, Yang (2026). *How Reliable Is Your Jailbreak Judge? Calibration and Adversarial Robustness of Automated ASR Scoring*. arXiv: [2606.25487 \[cs.CL\]](#).
- Liu, Xiaogeng et al. (2024). “AutoDAN: Generating Stealthy Jailbreak Prompts on Aligned Large Language Models”. In: *International Conference on Learning Representations (ICLR)*. arXiv: [2310.04451 \[cs.CL\]](#).
- Mei, Lingrui et al. (2025). ““Not Aligned” is Not “Malicious”: Being Careful about Hallucinations of Large Language Models’ Jailbreak”. In: *International Conference on Computational Linguistics (COLING)*. arXiv: [2406.11668 \[cs.CL\]](#).
- Weng, Shihao, Yang Feng, and Xiaofei Xie (2026). *Beyond Accuracy: Policy Invariance as a Reliability Test for LLM Safety Judges*. arXiv: [2605.06161 \[cs.CL\]](#).
- Yan, Yu et al. (2025). *Confusion is the Final Barrier: Rethinking Jailbreak Evaluation and Investigating the Real Misuse Threat of LLMs*. arXiv: [2508.16347 \[cs.CR\]](#).
- Yu, Jiahao et al. (2023). *GPTFUZZER: Red Teaming Large Language Models with Auto-Generated Jailbreak Prompts*. arXiv: [2309.10253 \[cs.CR\]](#).
- Yu, Mingyu et al. (2025). *AJF: Adaptive Jailbreak Framework Based on the Comprehension Ability of Black-Box Large Language Models*. arXiv: [2505.23404 \[cs.CR\]](#).

A Full Provenance Table

Table 5: Full per-claim provenance. Every number in the body with a row here is referenced via Section A at first mention.

Claim	File	Field(s)	Status
Qwen ours pool-12 ASR	results/ours_smoke_-pool12.json	asr, guided_fire_count, n_original_selected, n_mutated_child_selected, roberta_judge_positive_rate, trusted_judge_success_rate	Artifact-backed
Qwen gptfuzzer pool-12 ASR	results/gptfuzzer_smoke_-pool12.json	same fields	Artifact-backed
Phi ours pool-12 ASR	results/phi/ours_smoke_-pool12.json	same fields	Artifact-backed
Phi gptfuzzer pool-12 ASR	results/phi/gptfuzzer_-smoke_pool12.json	same fields	Artifact-backed
Qwen pre-fix ours ASR	results/ours_smoke.json	asr, guided_fire_count	Artifact-backed
Qwen direction AUC / layer	results/direction.npz	best_layer, best_layer_idx, best_auc	Artifact-backed
Phi direction AUC / layer	results/phi/direction.npz (mirrored results/phi4mini/direction.npz)	same fields	Artifact-backed
Qwen probe AUC / layer	results/probes/best_layer.json	best_layer, best_auc	Artifact-backed
Phi probe AUC / layer	results/phi/probes/best_layer.json (mirrored results/phi4mini/probes/best_layer.json)	same fields	Artifact-backed
Qwen direction/probe novel-prompt separation (30.204 / -4.954 / 0.500)	results/qwen_novel_check.log	per-prompt + mean direction_score lines, probe accuracy line	Artifact-backed (console log, not JSON, but committed and verbatim)
Phi direction/probe novel-prompt separation (9.486 / 4.594 / 0.500)	results/phi4mini/novel_check.log	same fields	Artifact-backed
ChadGPT false positive, true Qwen ours ASR 0.2	reviews/judge-validity-incident.md	prose	PI hand-verified, not written back into any JSON
Original Phi incident, 4/4 false positives, asr = 0.8	reviews/judge-validity-incident.md	prose	PI hand-verified (qualitative) / no backing JSON for the 0.8 figure
Pool-77 postfix Qwen 0.4/0.4	—	—	No backing file in this repository; PI-reported only
Section 5.3 attribution-quality examples + sum_score ranges (Qwen 40.0-116.0/82.6, Phi 51.7-87.2/65.8)	results/debug_attribution_qwen.log, results/debug_attribution_phi.log	top-10 tokens/selected span lines (assembled span text= field redacted; token/score/index fields unredacted), sum_score= values	Artifact-backed (console log, committed verbatim); superseded an earlier unbacked chat-excerpt claim
First MUTATED_CHILD selection at iteration 24 (Qwen)	results/debug_attribution_qwen.log	behavior_idx = 1 iteration-24 pool_select line	Artifact-backed

Claim	File	Field(s)	Status
First <code>MUTATED_CHILD</code> selection at iteration 24 (Phi)	<code>results/debug_attribution_phi.log</code>	<code>behavior_idx = 0</code> iteration-24 <code>pool_select</code> line	Artifact-backed; re-verified for this revision, previously reported in body text but not itself listed as a row here
<code>select_ucb1/backpropagate</code> tree-search-engagement proof (synthetic)	(verbatim-extracted, run standalone, not itself a committed artifact)	—	Reproducible from <code>scripts/run_fuzz.py</code> source; not a <code>results/*.json</code> fact; independently corroborated by the real iteration-24 log lines above, now for both targets